Final Group Project

ST443: Machine Learning & Data Mining

Candidate Numbers: 65191, 72033, 60602 & 66583

Department of Statistics

March 30, 2026

Abstract

This project applies statistical learning methods to two real-world classification tasks: multiclass land-type classification from hyperspectral satellite imagery and binary behavioral prediction from high-dimensional neural activity data. For Task 1, we analyzed 215,604 pixels across 220 spectral bands from an alpine region in Austria, comparing seven classification methods, including LDA, logistic regression, QDA, k-NN, xGBoost, Random Forest, and SVM, both with and without PCA dimensionality reduction. For Task 2, we addressed the challenge of predicting mouse directional choices from 11,190 neural features across only 683 trials. We implemented and compared five feature selection methods: Forward Stepwise Selection, LASSO, Random Forest variable importance, Elastic Net, and Boruta's algorithm. We worked to have effective handling of the high-dimensional, low-sample ($p >> n$) data. Our analysis revealed insights into optimal feature sizes, evidence of overfitting above certain thresholds, and more.

1 Task 1

1.1 Exploratory Data Analysis

The dataset has 215604 observations and 220 spectral bands, which differs from the project description of 218680 observations and 218 spectral bands. The observations correspond to the reflectance of each particular band. No *NA* values are observed.

Table 1: Class distribution of land cover types

Class	Count	Proportion
alpine meadow	50657	0.235
alpine tundra	38900	0.180
bare rock	26314	0.122
dark rock	10128	0.047
scree / sunlit rock	15153	0.070
snow / ice	11533	0.053
valley floor / meadow	38424	0.178
veg-scree mix	24495	0.114

There are 8 distinct land types. The most prevalent land type are alpine meadows, making up 23.5% of the land area. Dark rock is the least common land type, accounting for 4.70% of the land area. This would, to some extent, lead to a class imbalance factor of 5. We also observe that the snow/ice proportion is 5.3%. This may pose a problem for the subsequent glacier prediction task (1.4), as it becomes highly imbalanced.

In Figure 1, we plot a map of our dataset using the provided x- and y-coordinates. This serves as a useful cursory check for our data points. When compared with the provided map of the area, our map closely matches the actual image of the region. Next, we examine reflectance data across wavelength bands, stratified by land type.

As we can see in Figure 3, the mean spectral signatures reveal clear separability between certain land types. Snow and ice exhibit higher reflectance in the lower spectral bands. Land types with vegetation (alpine meadow, alpine tundra, valley floor, and veg-scree mix) exhibit similar reflectance patterns, suggesting they may be more challenging to discriminate. On the other hand, the different rock types exhibit more distinct spectral profiles, suggesting that their reflective features should provide stronger discriminative power for classification.

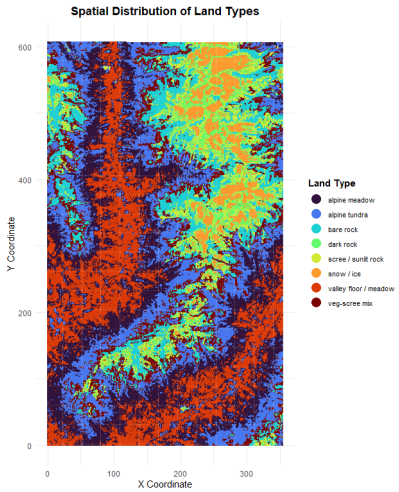


Figure 1: Land Type Spatial Visualisation

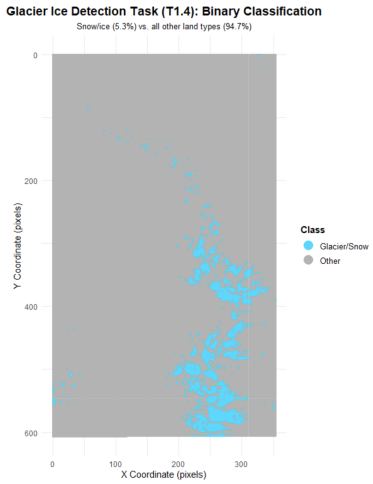


Figure 2: Glacier/Ice Class

Figure 4 plots the standard deviation of reflectance across different wavelength bands. We observe that for snow/ice land types, the standard deviation of reflectance increases substantially as we move toward the lower spectral bands. At lower bands, dark rock and scree/sunlit rock also exhibit a higher standard deviation of reflectance than other land masses. As we move to higher bands, the standard deviation of reflectance generally remains relatively low across all land types.

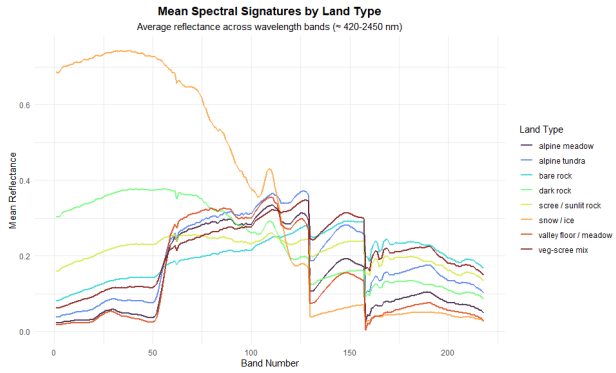


Figure 3: Average Reflectance Across Wavelength Bands

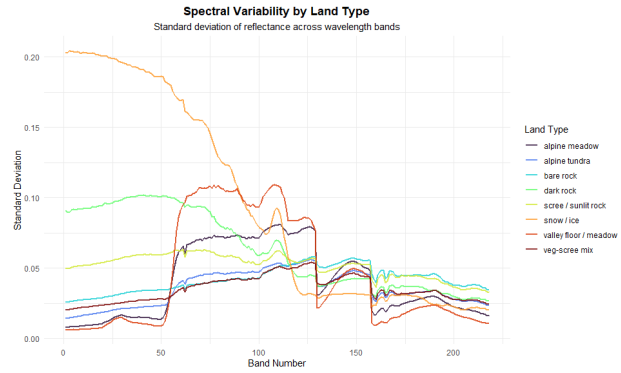


Figure 4: Standard deviation of reflectance across wavelength bands

1.2 Model Training and Evaluation

Before we begin the modeling process, we need to split our data into training and test sets. To do this, we use k-fold cross-validation. We split our data into 16 blocks, randomly selecting 3 blocks as the test set and 9 blocks as the training set. By splitting the image into non-overlapping spatial blocks, we ensure that training pixels are not adjacent to test pixels and that the test fold is spatially independent of the training folds.

From Table 2, the worst-performing model is the SVM (Original), which has the lowest F1-score (0.637) and balanced accuracy (0.780). This occurs because SVMs perform poorly on high-dimensional, highly correlated hyperspectral data unless their parameters are carefully tuned. However, we were unable to improve model accuracy beyond this level, even after tuning our hyperparameters via grid

search. Furthermore, we used only 10% of the training data to train the model due to computational constraints. Without PCA, the SVM must work with many noisy and redundant bands, leading to overfitting and poor decision boundaries. Although its performance improves after PCA, the SVM still remains one of the weaker models in our comparison.

Table 2: Summary of evaluation results

Model	Accuracy	Error Rate	Bal. Acc	AUC	F1
LDA (Original)	0.840	0.160	0.901	0.988	0.838
LDA (PCA)	0.830	0.170	0.895	0.987	0.831
Logistic (Original)	0.946	0.054	0.963	0.998	0.931
Logistic (PCA)	0.973	0.027	0.987	**1.000**	0.978
QDA (Original)	0.812	0.188	0.880	0.979	0.794
QDA (PCA)	0.955	0.045	0.976	0.998	0.957
k-NN (Original)	0.964	0.036	0.978	0.999	0.961
k-NN (PCA)	**0.979**	**0.021**	**0.988**	**1.000**	**0.979**
xGBoost (Original)	0.963	0.037	0.978	0.999	0.961
xGBoost (PCA)	0.973	0.027	0.985	**1.000**	0.975
RF (Original)	0.965	0.035	0.978	0.999	0.962
RF (PCA)	0.972	0.028	0.984	0.999	0.973
SVM (Original)	0.705	0.295	0.780	0.985	0.637
SVM (PCA)	0.719	0.281	0.816	0.989	0.737

The best-performing models in our multi-class land-type classification task are Logistic Regression (PCA) and k-NN (PCA), both achieving very high F1-scores (around 0.97-0.98) and near-perfect AUC values. These models perform well because PCA significantly simplifies hyperspectral data by removing noise and reducing strong interband correlations. This makes the classes much easier to separate. Logistic regression benefits from this because the reduced feature space is more linearly separable. k-NN works better because distances between points become more meaningful after dimensionality reduction. Overall, PCA helps reveal the main structure in the data, allowing these otherwise simple models to achieve excellent performance.

1.3 Classification Performance

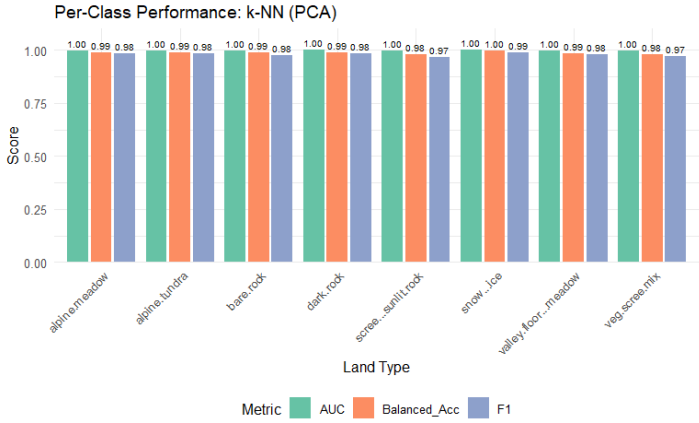


Figure 5: kNN with PCA Classification Performance

We break down the performance of our top model to visualise per-class classification. In Figure 5, we see that accuracy is generally good across all classes. In particular, we achieved a balanced accuracy score of nearly 100% and an F1-score of 99% for the snow/ice category, which would be useful for the next glacier classification task.

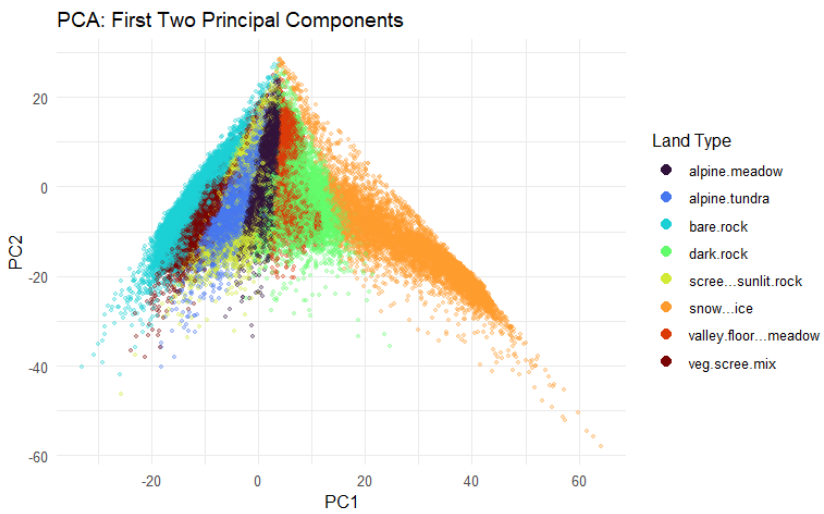


Figure 6: PC1 and PC2 in k-NN Model with PCA

We can also visualise the PCA components to assess the separation achieved by the first 2 PCs. This plot is useful because the first 2 PCs account for about 80.7% of the variance. We observe generally good separation across the classes, except for the scree/sunlit rock class. However, this should be captured in the other PCs, given that the model achieved such a high accuracy score.

1.4 Glacier Classification

In our exploratory data analysis, we observed that the snow/ice class was the second-smallest class. The class imbalance (5.3% positive class) suggests that evaluation metrics such as F1-score will be more appropriate than overall accuracy, as a naive classifier predicting "non-glacier" for all pixels would achieve 94.7% accuracy while being completely uninformative. We ran three classifiers to improve our F1 score each time. Since glacier pixels are spatially clustered and sparse within the grid (see Figure 2), we used random k-fold cross-validation rather than a spatial grid-based split, ensuring that each fold contained at least some positive examples and preventing folds with no glacier pixels.

The first model we ran was LDA with PCA. LDA assumes that each class follows a multivariate normal distribution with a shared covariance matrix. Our data show that spectral bands are often non-normally distributed, and the covariance structure of glacier pixels differs noticeably from that of non-glacier surfaces. Since our data violates these LDA assumptions, we expected LDA to perform relatively poorly. However, our LDA model still achieved a strong mean F1-score of 0.93 across 5 folds, indicating that the glacier and non-glacier classes are sufficiently separable in the feature space for a linear boundary to perform well. Next, we tried a logistic regression model, as it does not assume normality and is therefore likely to perform better than the LDA model. We also ran the model with PCA. Logistic regression achieved an exceptionally high average F1-score of 0.998 across 5 folds, indicating that a linear boundary is highly effective for separating glacier from non-glacier pixels in this dataset. Lastly, we modeled a random forest. We expected this model to achieve the highest F-1 score as random forests are well-suited to hyperspectral classification because they can model

nonlinear relationships between spectral bands and capture complex interactions that linear methods may miss. The model achieved an average F1-score of 0.991 across 5 folds, which is high but slightly lower than that of our logistic regression model.

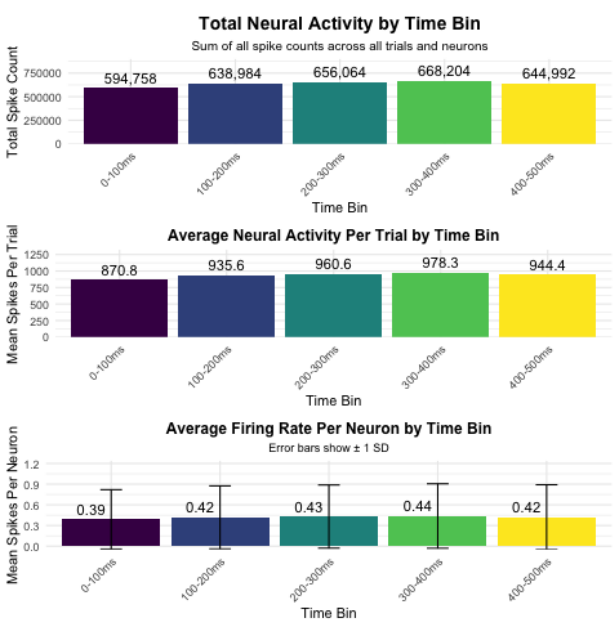
Logistic regression outperformed the random forest by a small margin, likely because the glacier vs. non-glacier classes in this dataset are almost linearly separable in spectral space. Because it is more flexible, the random forest can overfit noise in individual spectral bands and approximate linear boundaries less efficiently. As a result, logistic regression captured the underlying separation more cleanly, yielding the highest F1-score.

2 Task 2

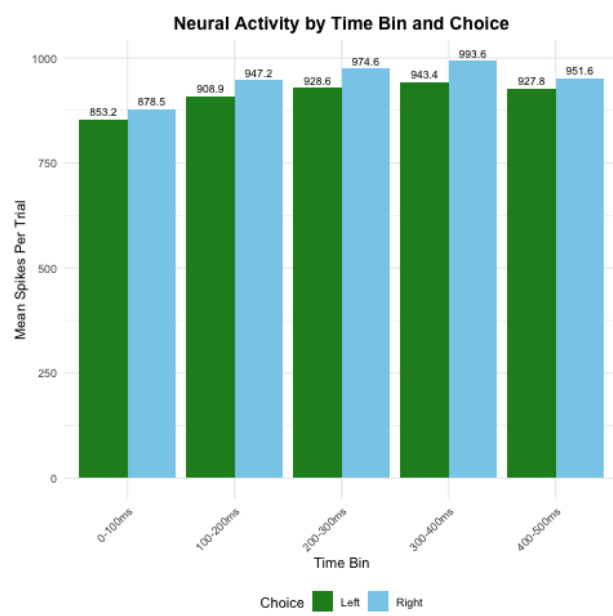
2.1 Exploratory Data Analysis

The dataset has 683 observations across 11191 variables. Each observation represents a trial in the experiment. The variables consist of 1 binary direction indicator and 2238 neurons measured over 5 time bins of 100 ms each. The dataset is complete; no *NA* values are observed. We are using the dataset to predict mouse movement. The *label* variable is binary, with 0 indicating left and 1 indicating right; there are 208 observations of the mouse moving left and 475 of it moving right.

Figure 7a shows the total and average neuron activity by time bin, showing that the neurons increase their firing over time, peaking at 300-400 ms. We observe this trend at the level of individual neurons, with the last barplot in Figure 7a showing that the average firing rate per neuron is highest during the same time period. Figure 7b shows the average neural activity separated by directional choice; the same trend persists in this figure, too. We also observe that the average neural firing rate is lower for a mouse moving left than for one moving right; this is present in every time bin throughout the experiment.



(a) Tiled barplots displaying different neural activity by time bin.



(b) A barplot displaying average spikes by time bin for mice choosing left or right.

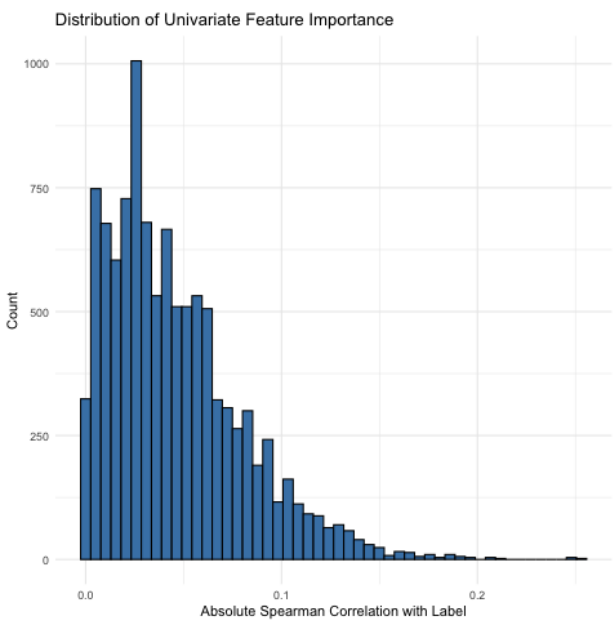
Figure 7

2.2 Model Selection, Training & Evaluation

2.2.1 Forward Stepwise (Subset) Selection

Forward Stepwise Selection is an algorithm that builds a model by iteratively adding one feature at a time. The algorithm starts with no features and adds the feature that most improves the model according to a specified criterion. We found that we could not run this algorithm without modifications due to its computational intensity. First, we selected feature subsets for the algorithm and iteratively evaluated each subset of n features for accuracy. Second, we ranked each feature by the magnitude of the Spearman correlation with the *label* feature. This practice makes it easier to select subsets and improve model accuracy by adding important features. We used Spearman's rank correlation for our ranking because the data were not normally distributed; Figure 8a shows the distribution of feature correlation magnitudes. For each subset size, we evaluated the model using cross-validated balanced accuracy, giving us performance curves for models with n features; we used the following subset sizes:

$$n \in \{1, 2, 5, 10, 15, 25, 50, 75, 100, 125, 150, 200, 300, 500, 750, 1000, 1500, 2000, 3000, 5000\}.$$



(a) A frequency histogram of the distribution of the correlations of each feature.

(b) A plot of training and testing performance over different subsets of n features.

Figure 8

We saw that as n increased, our test performance did not always increase. In fact, as seen in Figure 8b, test performance peaked at 100 features with 77.85% balanced accuracy, then proceeded to sharply fall; there is a group of n's (75, 100, 125, 150, 200) that achieve over 75%. Once $n \geq 750$, balanced accuracy falls past 60%. This sharp decrease in accuracy is due to overfitting. The 100% test accuracy confirms this, observed for $n \geq 300$. While not exactly the Forward Stepwise Selection process, we use this modified version because it is computationally feasible for our high-dimensional dataset and yields a reasonable approximation.

2.2.2 LASSO

LASSO regression is a regularized version of logistic regression, adding an *L1* penalty to each coefficient. For a binary outcome, the LASSO logistic model minimizes the penalized negative log-likelihood

$$\text{Negative log likelihood} + \lambda \sum_{j=1}^{p} |\beta_j|$$

where $\lambda \geq 0$ controls strength of regularization. The penalty term $\lambda \sum_{j=1}^{p} |\beta_j|$ shrinks all coefficients toward zero. With a large λ, some β_j are exactly zero, so the model performs feature selection as part of fitting.

In this study, the training set was used to fit a LASSO logistic regression model over a grid of λ values, and 5-fold cross-validation was used to select the λ that minimized classification error. Figure 9 shows the 5-fold CV error vs. each value of λ. λ_{min} shows the best CV error, λ_{1se} would create a more regularized, sparser model. We then fit a standard logistic regression to the selected features using λ_{min}, as our goal is to maximize balanced accuracy.

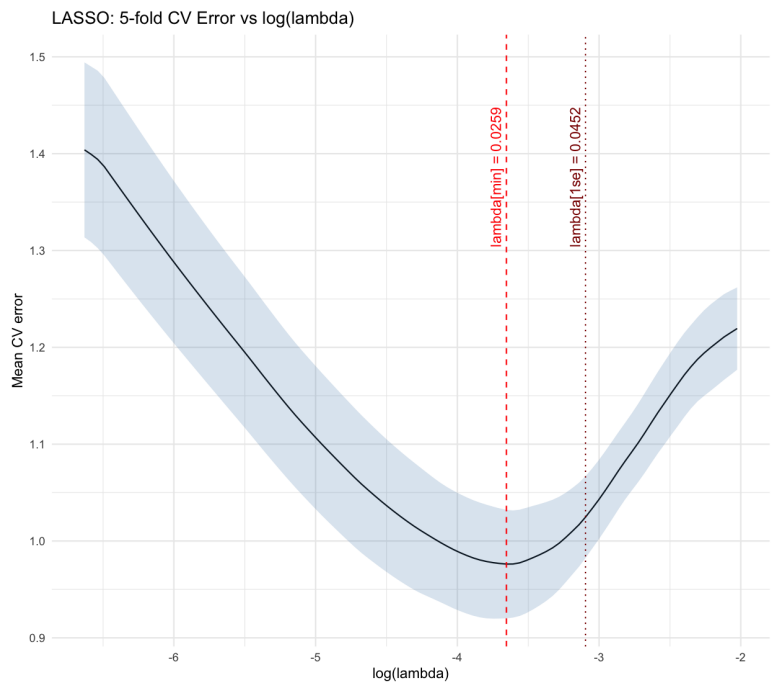


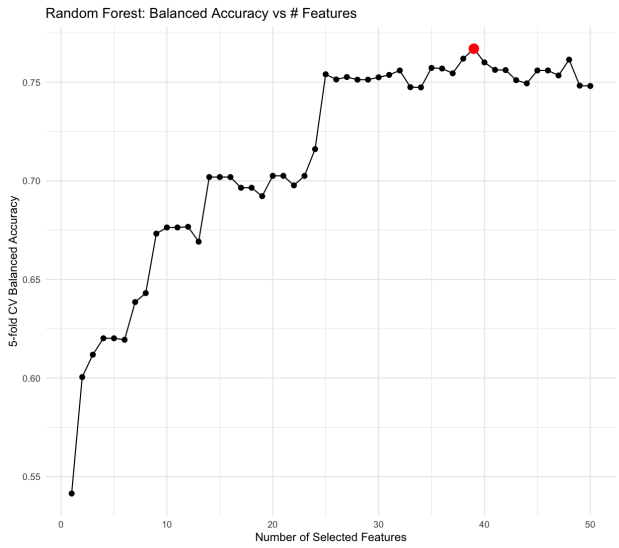
Figure 9: Values of λ for the LASSO regression model.

2.2.3 Random Forest

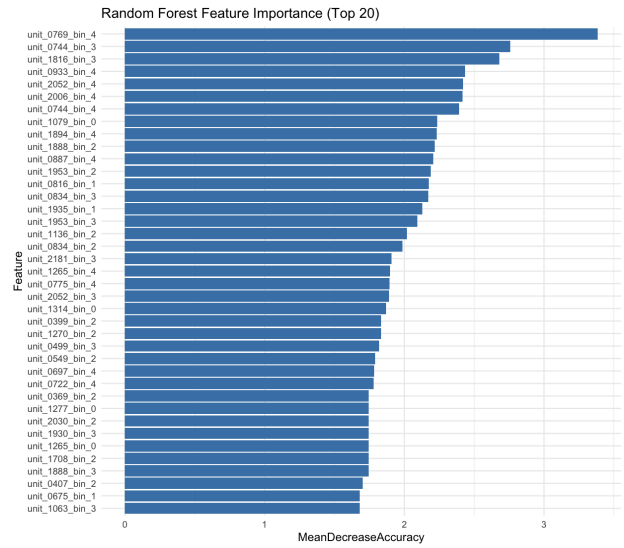
Random forests extend bagged trees by decorrelating individual trees, further reducing variance when their predictions are averaged. Tree-based methods partition the feature space via recursive splits, while bagging fits multiple trees on bootstrapped samples of the training data and averages their predictions. In random forests, each split is chosen from a random subset of m predictors (typically $m \approx \sqrt{p}$ for classification) rather than all p predictors, which encourages diversity among trees.

In this study, random forests are used as a nonlinear classifier and as a feature-ranking method via mean decrease in accuracy. This gives us a ranking of neuron–time-bin predictors by their contribution to predictive performance. We trained logistic regression models on the top k RF-ranked features for $k = 1, \ldots, 50$. We then performed 5-fold cross-validation to compute the model's balanced accuracy for each k. Figure 10a plots balanced accuracy against the number of selected features, with the best-performing k highlighted in red. Performance improves steeply up to around 25–30 features, then

plateaus, suggesting that only 30–40 features are sufficient to achieve near-maximal balanced accuracy while keeping the model compact.



(a) A plot of CV balanced accuracy over the various top-k features using Random Forest



(b) Histogram of importance of the 39 features selected from the best RF model.

Figure 10

Figure 10b shows the top 39 features by Random Forest importance. The most informative predictors are a small subset of neurons, mostly from later time bins, suggesting that decision-related information is concentrated in specific cells and late post-stimulus activity. We then fit a standard logistic regression on the best features and calculated full metrics.

2.2.4 Boruta's Algorithm

Boruta is a feature selection method that builds on top of a tree-based model (most commonly a random forest). However, instead of directly selecting a minimal subset of predictors, it tests each feature to determine whether its importance is significantly higher than that of a "shadow" feature (a random, noise-like feature), retaining all features that pass this relevance test.

Boruta works through iterations. In each iteration, it creates "shadow" features by randomly shuffling the values of each original predictor. This destroys any meaningful association with the response while preserving marginal distributions. It then trains a random forest, like we did in 2.2.3, but with a dataset containing both original and shadow features.. For each real feature, Boruta computes its importance and compares it to the maximum importance among the shadow features. If a real feature is consistently lower, it is labeled "Rejected," if it is consistently higher, it is labeled "Confirmed." Otherwise, it remains "Tentative" until its status is resolved in a subsequent iteration (or the maximum iteration limit is reached).

We ran Boruta on our training data, repeatedly fitting random forests to the original neuron–time-bin features augmented with shadow features and tracking importance scores across iterations. We set parameters of $maxRows = 50$ and $pValue = 0.05$. The former sets the maximum number of Boruta iterations, capping iterations at 50, which helps us balance model accuracy with computational cost. The latter is the significance threshold used to compare importance across shadow features. Real features whose importance is consistently higher than the best shadow at this 5% level will be marked

as confirmed, while those consistently worse are rejected. Borderline cases will remain tentative when the algorithm stops.

Ideally, these settings will allow us to identify the subset of neuron–time-bin features whose importance is reliably greater than that expected from random noise. These will also be more relevant for predicting the mouse's direction of motion. We then fit a standard logistic regression to two feature subsets—one with only the confirmed features, and one with both confirmed and tentative features. We then calculated full metrics for each model.

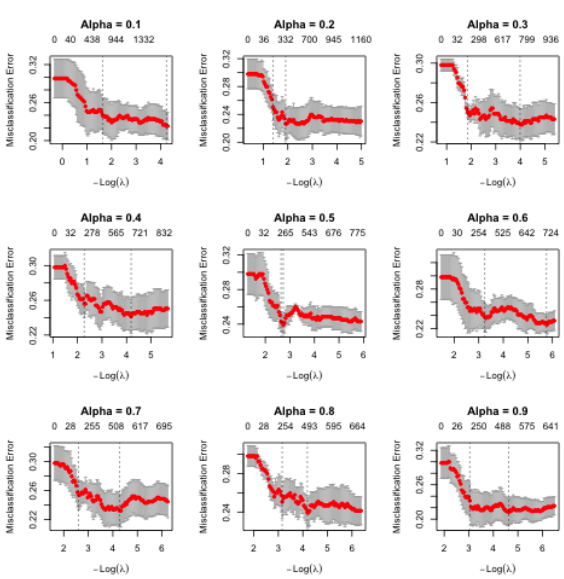
2.2.5 Elastic Net

Elastic Net is a regularization method that combines the *L1* penalty (LASSO) with the *L2* penalty (Ridge Regression). While LASSO performs feature selection by shrinking coefficients to 0, it can become unstable when features are highly correlated, which can lead the algorithm to arbitrarily select a feature from a group. Elastic Net reduces this issue by introducing the Ridge penalty, which encourages grouped selection of correlated features. The Elastic Net penalty is

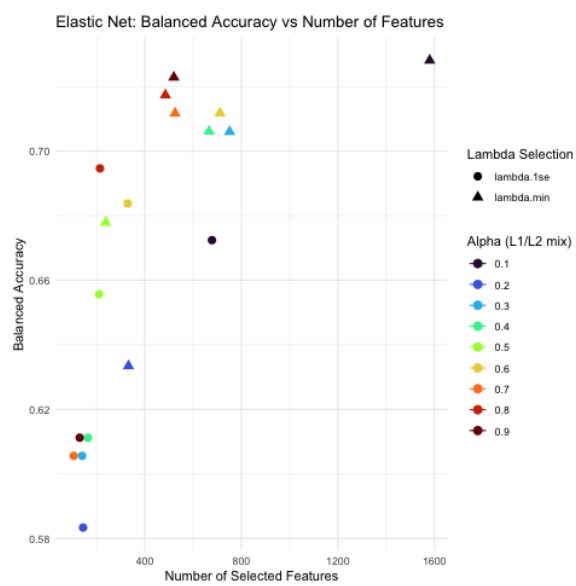
$$\lambda \times [\alpha \times ||\beta||_1 + (1-\alpha) \times ||\beta||_2^2],$$

where α is the penalty mixture parameter and λ is the regularization strength parameter. The closer α is to 1, the more LASSO (β_1); the closer it is to 0, the more Ridge we have (β_2).

For our neuronal data, this is advantageous because we can assume that neuronal activity is correlated across adjacent time bins or across similar regions of the mouse brain. Furthermore, because we have many features relative to observations ($p >> n$), many models cannot handle this dimensionality. However, each piece of Elastic Net can help to manage this. The LASSO model will help us reduce dimensionality through feature selection; Ridge will help with overfitting and noise reduction, and improve stability. Each piece of the model works in tandem to ensure precise, accurate, and stable approximations.



(a) Plots of misclassification errors over the $-\log$ of different λ's for various α's.



(b) A scatterplot of balanced accuracy over n features for α's with $\lambda_{\min}$ or λ_{1SE}.

Figure 11

In our study, we evaluated performance across different values of α to determine the optimal LASSO-to-Ridge ratio. Figure 11a plots the misclassification errors for different α's; we use a 5-fold cross-validation to find the optimal λ. Each plot has two vertical dotted gray lines: the left one is the lambda that provides the least CV error ($\lambda_{\min}$) and the right is the largest lambda within 1 standard error of $\lambda_{\min}$, or λ_{1SE}. Further, $\lambda_{\min}$ provides the best CV performance, but overfitting can be an issue; λ_{1SE} negates the overfitting problem, but is a simpler model and has a lower CV accuracy.

These are the values of λ that we use to further evaluate variations of our model in Figure 11b, which shows the balanced accuracy over the number of selected features based on different values of α and choices in λ. We observe a logarithmic trend in the points: as the number of selected features increases, the accuracy increases at a logarithmic rate. We also see that a smaller λ always yields a higher balanced accuracy. This makes sense because a model with a larger $\lambda_{\min}$ will include more features in its predictions and, barring overfitting, will produce more accurate predictions. We observe that the model with the highest balanced accuracy is obtained with $\alpha = 0.1$ and $\lambda = \lambda_{\min}$.

2.2.6 Results

Ultimately, we evaluated five feature selection methods on our dataset. The results from each method can be found in Table 3. Each of our methods has distinct advantages and drawbacks, with some being particularly computationally expensive or struggling to achieve at least 75% accuracy. We observe that most of our methods were within 15% of one another across the various statistics.

Method	Num Features	Accuracy	Bal. Acc	AUC	F1
LASSO	246	0.7647	0.6838	0.8669	0.8400
RF Top-k	39	0.7353	0.6562	0.7971	0.8182
Forward Stepwise	75	0.7647	0.7062	0.8105	0.8333
Elastic Net	832	**0.8309**	**0.7669**	**0.9094**	**0.8832**
Boruta	6	0.7132	0.6004	0.6988	0.8134
Boruta (with Tentative)	46	0.7426	0.6841	0.8037	0.8168

Table 3: Task 2 Feature Selection Results. Bold values indicate the best performance across methods.

Of all of our methods, **Elastic Net** had the best statistics. However, this came at a cost: the method required 832 features to achieve it. This reveals the greedy but accurate nature of the Elastic Net algorithm. In contrast, LASSO's accuracy was only 5% lower than Elastic Net's, but it required fewer than 30% of the features to achieve this. Interestingly, Elastic Net uses LASSO within its model; however, this version of Elastic Net uses only 10% LASSO and 90% Ridge regression.

Part of the reason we selected the five methods was that we found the relationship between Random Forest/Boruta's Algorithm and LASSO/Elastic Net particularly interesting. Sometimes, like we see with Random Forest Top-k and Boruta (without tentative) in Table 3, more advanced is not necessarily better for accuracy. The results of these methods reveal something most of us already know to be true: some models perform better on some data than others. The privilege of choice gives us the freedom to decide how to analyze our data based on its characteristics.

References

Anthropic. *Claude*. Anthropic, https://claude.ai . Accessed 11 Dec. 2025.

"Cv.glmnet Function - RDocumentation." Rdocumentation.org, 2025, www.rdocumentation.org/packages/glmnet/versions/4.1-10/topics/cv.glmnet.

Kursa, M. B., & Rudnicki, W. R. (2010). Feature Selection with the Boruta Package. *Journal of Statistical Software*, 36(11), 1–13. https://doi.org/10.18637/jss.v036.i11

Perplexity AI. *Perplexity*. Perplexity AI, https://www.perplexity.ai . Accessed 11 Dec. 2025.